

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

Національний аерокосмічний університет

Факультет систем управління літальних апаратів

Кафедра систем управління літальних апаратів

XAI.319. G7 Автоматизація, комп'ютерно-інтегровані технології та
робототехніка. 319. 38 ЛР

Виконав студент група 319

Олександр *Ткачук*

23.03.2026

Лабораторна робота No9 (методичні вказівки)

з дисципліни «Алгоритмізація процесів управління»

Тема: "Реалізація алгоритмів сортування та робота з файлами на мові C ++"

МЕТА РОБОТИ

Вивчити теоретичний матеріал з основ роботи з низькорівневими рядками на C++ і документацію до класу string, а також алгоритми пошуку в рядку, а

також реалізувати обробку рядків на C++ в середовищі QtCreator.

ПОСТАНОВКА ЗАДАЧІ

Завдання 1.

А. Вивчити по документації метод стандартного класу `string` відповідно до варіанту (див. табл.1).

В. Визначити функцію, що виконує ті ж дії, що і вивчений метод класу `string`. Вихідний рядок передати першим параметром (масив символів).

Для реалізації методу не використовувати функції обробки рядків зі стандартних бібліотек.

С. Викликати свій метод і метод `string` аналогічно прикладам коду, наведеними в дод.А. *Перед викликом ввести з консолі один рядок і зберегти в масиві символів і змінній типу `string`.

Завдання 2.

А.Описати функцію, що обробляє рядок відповідно до завдання з табл.2. Для реалізації можна використовувати функції обробки рядків зі стандартних бібліотек

В.Описати функцію, яка перевіряє, чи задовольняє рядок умовам завдання.

С.* Створити вихідний текстовий файл, що містить не менше 10 різних рядків.

Д.Використовуючи функції 2.А і 2.В, обробити рядок / * текстовий файл рядок за рядком. Додаткові дані ввести з консолі.

Е. Отриманий результат записати у вихідний файл.

Завдання 3.

Завдання 1-2 реалізувати окремими функціями без параметрів, у функції `main()` організувати меню для багаторазового виконання завдань.

Структурувати проєкт програми: винести заголовки і реалізацію функцій в окремі `.h` та `.cpp` файли.

Таблиця 1 – Варіанти методів `string`

39. `size_t find_last_of(char c, size_t pos = npos) const;`

String50. Дано рядок, що складається з кириличних слів, розділених пробілами (одним або декількома). Вивести рядок, що містить ці ж слова, розділені одним пробілом і розташовані в зворотному порядку.

Лістинг коду програми

```
Lab9 (main)
#include <iostream>

#include "task1.h"
#include "task2.h"

using namespace std;

void showMenu() {
    cout << "\n===== MENU =====\n";
    cout << "1 - Task 1\n";
    cout << "2 - Task 2\n";
    cout << "0 - Exit\n";
    cout << "Choose: ";
}

int main() {
    int choice;

    do {
        showMenu();
        cin >> choice;

        switch (choice) {
            case 1:
                task1();
                break;
            case 2:
                task2();
                break;
            case 0:
```

```

        cout << "Program finished.\n";

        break;

    default:

        cout << "Wrong menu item!\n";

    }

    } while (choice != 0);

    return 0;
}

Task1.h
#ifndef TASK1_H
#define TASK1_H

#include <string>

int myFindLastOf(const char s[], char c);
void task1();

#endif

Task1.cpp
#include "task1.h"
#include <iostream>
#include <string>
#include <limits>

using namespace std;

// Власна функція пошуку останнього входження символу в char[]
int myFindLastOf(const char s[], char c) {
    int lastPos = -1;

    for (int i = 0; s[i] != '\0'; i++) {
        if (s[i] == c) {
            lastPos = i;
        }
    }
}

```

```

    }

    return lastPos;
}

void task1() {
    const int MAX_LEN = 256;
    char cstr[MAX_LEN];
    string str;
    char target;

    cin.ignore(numeric_limits<streamsize>::max(), '\n');

    cout << "==== TASK 1 =====\n";
    cout << "Enter a string: ";
    cin.getline(cstr, MAX_LEN);

    str = cstr;

    cout << "Enter a symbol to search: ";
    cin >> target;

    // Власна функція
    int myPos = myFindLastOf(cstr, target);

    // Метод string
    size_t stdPos = str.find_last_of(target);

    cout << "\nResult of myFindLastOf(): ";
    if (myPos == -1)
        cout << "symbol not found\n";
    else
        cout << "last position = " << myPos << endl;

    cout << "Result of string::find_last_of(): ";
    if (stdPos == string::npos)

```

```

        cout << "symbol not found\n";
    else
        cout << "last position = " << stdPos << endl;
}

Task2.h*
#ifndef TASK2_H
#define TASK2_H

#include <string>

bool isValidString51(const std::string& s);
std::string processString51(const std::string& s);
void task2();

#endif#pragma once

Task2.cpp
#include "task2.h"
#include <iostream>
#include <fstream>
#include <string>
#include <vector>
#include <algorithm>
#include <limits>

using namespace std;

// Перевірка: чи символ є пробілом
bool isSpaceChar(char ch) {
    return ch == ' ';
}

// Перевірка для простого варіанта:
// дозволяємо пробіл і байти кирилиці у Windows-1251 / ANSI.
// Для навчальної лабораторної цього достатньо.
bool isUpperCyrillicChar(unsigned char ch) {
    return (ch >= 192 && ch <= 223) || ch == 168; // А-Я, Ё

```

```

}

// Перевірка рядка за умовою String51
bool isValidString51(const string& s) {
    if (s.empty()) return false;

    bool hasLetter = false;

    for (size_t i = 0; i < s.size(); i++) {
        unsigned char ch = static_cast<unsigned char>(s[i]);

        if (isSpaceChar(ch)) {
            continue;
        }

        if (!isUpperCyrillicChar(ch)) {
            return false;
        }

        hasLetter = true;
    }

    return hasLetter;
}

// Обробка String51:
// 1) розбиття на слова
// 2) сортування
// 3) складання через один пробіл
string processString51(const string& s) {
    vector<string> words;
    string current = "";

    // Розбити на слова, ігноруючи зайві пробіли
    for (size_t i = 0; i < s.size(); i++) {
        if (s[i] != ' ') {

```

```

        current += s[i];
    }
    else {
        if (!current.empty()) {
            words.push_back(current);
            current.clear();
        }
    }
}

if (!current.empty()) {
    words.push_back(current);
}

// Сортування за алфавітом
sort(words.begin(), words.end());

// Зібрати рядок назад через один пробіл
string result = "";
for (size_t i = 0; i < words.size(); i++) {
    result += words[i];
    if (i + 1 < words.size()) {
        result += ' ';
    }
}

return result;
}

void task2() {
    int choice;

    cout << "==== TASK 2 =====\n";
    cout << "1 - Process one line from console\n";
    cout << "2 - Process file line by line\n";
    cout << "Choose: ";

```



```

cin >> choice;

cin.ignore(numeric_limits<streamsize>::max(), '\n');

if (choice == 1) {
    string s;
    cout << "Enter a string (uppercase Cyrillic words): ";
    getline(cin, s);

    if (!isValidString51(s)) {
        cout << "Wrong string! Only uppercase Cyrillic words and spaces are
allowed.\n";
        return;
    }

    string result = processString51(s);
    cout << "Result: " << result << endl;
}

else if (choice == 2) {
    string inputFile, outputFile;
    cout << "Enter input file name: ";
    getline(cin, inputFile);
    cout << "Enter output file name: ";
    getline(cin, outputFile);

    ifstream fin(inputFile);
    if (!fin) {
        cout << "Cannot open input file!\n";
        return;
    }

    ofstream fout(outputFile);
    if (!fout) {
        cout << "Cannot open output file!\n";
        fin.close();
        return;
    }
}

```

```
}

string line;
while (getline(fin, line)) {
    if (isValidString51(line)) {
        fout << processString51(line) << endl;
    }
    else {
        fout << "INVALID STRING" << endl;
    }
}

fin.close();
fout.close();

cout << "File processed successfully.\n";
}
else {
    cout << "Wrong menu item!\n";
}
}
```

```
===== MENU =====
```

```
1 - Task 1
```

```
2 - Task 2
```

```
0 - Exit
```

```
Choose: 1
```

```
===== TASK 1 =====
```

```
Enter a string: panorama
```

```
Enter a symbol to search: m
```

```
Result of myFindLastOf(): last position = 6
```

```
Result of string::find_last_of(): last position = 6
```

```
===== MENU =====
```

```
1 - Task 1
```

```
2 - Task 2
```

```
0 - Exit
```

```
Choose: 1
```

```
===== TASK 1 =====
```

```
Enter a string: labrador
```

```
Enter a symbol to search: l
```

```
Result of myFindLastOf(): last position = 3
```

```
Result of string::find_last_of(): last position = 3
```

```
===== MENU =====
```

```
1 - Task 1
```

```
2 - Task 2
```

```
0 - Exit
```

```
Choose: |
```

```
===== MENU =====
```

```
1 - Task 1
```

```
2 - Task 2
```

```
0 - Exit
```

```
Choose: 2
```

```
===== TASK 2 =====
```

```
1 - Process one line from console
```

```
2 - Process file line by line
```

```
Choose: 2
```

```
Enter input file name: file.txt
```

```
Enter output file name: file2.txt
```

```
File processed successfully.
```

```
===== MENU =====
```

```
1 - Task 1
```

```
2 - Task 2
```

```
0 - Exit
```

```
Choose: |
```





